

Sales Prediction Dataset Analysis

Importing Dataset

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

dataset = pd.read_excel("advertising_sales_data.xlsx")
print("Full Dataset Overview -")
print(dataset)
print("-----")
dataset.head()
dataset.info()
print("-----")
dataset.isna().sum()
```

Full Dataset Overview -

	Campaign	TV	Radio	Newspaper	Sales
0	camp1	230.1	37.8	69.2	22.1
1	camp2	44.5	39.3	45.1	10.4
2	camp3	17.2	45.9	69.3	12.0
3	camp4	151.5	41.3	58.5	16.5
4	camp5	180.8	10.8	58.4	17.9
..	...	...	...	...	...
195	camp196	38.2	3.7	13.8	7.6
196	camp197	94.2	4.9	8.1	14.0
197	camp198	177.0	9.3	6.4	14.8
198	camp199	283.6	42.0	66.2	25.5
199	camp200	232.1	8.6	8.7	18.4

[200 rows x 5 columns]

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Campaign    200 non-null   object
1   TV          200 non-null   float64
2   Radio       198 non-null   float64
3   Newspaper   200 non-null   float64
4   Sales       200 non-null   float64
dtypes: float64(4), object(1)
memory usage: 7.9+ KB
-----
```

```
Campaign    0
TV          0
Radio       2
Newspaper   0
Sales       0
dtype: int64
```

Cleaning Dataset (Missing values, Duplicates)

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="mean")
dataset[["Radio"]] = imputer.fit_transform(dataset[["Radio"]])

print(dataset.isna().sum())
dataset.info()

Campaign    0
TV          0
Radio       0
Newspaper   0
Sales       0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Campaign    200 non-null    object
 1   TV          200 non-null    float64
 2   Radio       200 non-null    float64
 3   Newspaper   200 non-null    float64
 4   Sales       200 non-null    float64
dtypes: float64(4), object(1)
memory usage: 7.9+ KB
```

Encoding Categorical Data

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(transformers=[('encoder',
OneHotEncoder(drop="first"), ["Campaign"])], remainder='passthrough')
X= ct.fit_transform(dataset)
print(X)

<Compressed Sparse Row sparse matrix of dtype 'float64'
  with 998 stored elements and shape (200, 203)>
  Coords  Values
(0, 199) 230.1
```

(0, 200)	37.8
(0, 201)	69.2
(0, 202)	22.1
(1, 110)	1.0
(1, 199)	44.5
(1, 200)	39.3
(1, 201)	45.1
(1, 202)	10.4
(2, 122)	1.0
(2, 199)	17.2
(2, 200)	45.9
(2, 201)	69.3
(2, 202)	12.0
(3, 133)	1.0
(3, 199)	151.5
(3, 200)	41.3
(3, 201)	58.5
(3, 202)	16.5
(4, 144)	1.0
(4, 199)	180.8
(4, 200)	10.8
(4, 201)	58.4
(4, 202)	17.9
(5, 155)	1.0
:	:
(195, 106)	1.0
(195, 199)	38.2
(195, 200)	3.7
(195, 201)	13.8
(195, 202)	7.6
(196, 107)	1.0
(196, 199)	94.2
(196, 200)	4.9
(196, 201)	8.1
(196, 202)	14.0
(197, 108)	1.0
(197, 199)	177.0
(197, 200)	9.3
(197, 201)	6.4
(197, 202)	14.8
(198, 109)	1.0
(198, 199)	283.6
(198, 200)	42.0
(198, 201)	66.2
(198, 202)	25.5
(199, 112)	1.0
(199, 199)	232.1
(199, 200)	8.6

(199, 201)	8.7
(199, 202)	18.4

Features Defining -

```
X = dataset[["TV", "Radio", "Newspaper"]]
y = dataset["Sales"]
```

Data Splitting -

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size =
0.2, random_state = 42)
print(x_train)
print(x_test)
print(y_train)
print(y_test)

print("Training set size:", x_train.shape[0], "rows")
print("Testing set size:", x_test.shape[0], "rows")
```

	TV	Radio	Newspaper
79	116.0	7.7	23.1
197	177.0	9.3	6.4
38	43.1	26.7	35.1
24	62.3	12.6	18.3
122	224.0	2.4	15.6
..	...	...	...
106	25.0	11.0	29.7
14	204.1	32.9	46.0
92	217.7	33.5	59.0
179	165.6	10.0	17.6
102	280.2	10.1	21.4

[160 rows x 3 columns]			
	TV	Radio	Newspaper
95	163.3	31.600000	52.9
15	195.4	47.700000	52.9
30	292.9	28.300000	43.2
158	11.7	36.900000	45.2
128	220.3	49.000000	3.2
115	75.1	35.000000	52.7
69	216.8	43.900000	27.2
170	50.0	11.600000	18.4
174	222.4	3.400000	13.1
45	175.1	22.500000	31.5
66	31.5	24.600000	2.2
182	56.2	5.700000	29.7
165	234.5	3.400000	84.8

78	5.4	29.900000	9.4
186	139.5	2.100000	26.6
177	170.2	7.800000	35.2
56	7.3	28.100000	41.4
152	197.6	23.260606	14.2
82	75.3	20.300000	32.5
68	237.4	27.500000	11.0
124	229.5	32.300000	74.2
16	67.8	36.600000	114.0
148	38.0	40.300000	11.9
93	250.9	36.500000	72.3
65	69.0	9.300000	0.9
60	53.5	2.000000	21.4
84	213.5	43.000000	33.8
67	139.3	14.500000	10.2
125	87.2	11.800000	25.9
132	8.4	27.200000	2.1
9	199.8	2.600000	21.2
18	69.2	20.500000	18.3
55	198.9	49.400000	60.0
75	16.9	43.700000	89.4
150	280.7	13.900000	37.0
104	238.2	34.300000	5.3
135	48.3	47.000000	8.5
137	273.7	28.900000	59.7
164	117.2	14.700000	5.4
76	27.5	1.600000	20.7
79	11.0		
197	14.8		
38	10.1		
24	9.7		
122	16.6		
	...		
106	7.2		
14	19.0		
92	19.4		
179	17.6		
102	19.8		
Name: Sales, Length: 160, dtype: float64			
95	16.9		
15	22.4		
30	21.4		
158	7.3		
128	24.7		
115	12.6		
69	22.3		
170	8.4		
174	16.5		
45	16.1		

66	11.0
182	8.7
165	16.9
78	5.3
186	10.3
177	16.7
56	5.5
152	16.6
82	11.3
68	18.9
124	19.7
16	12.5
148	10.9
93	22.2
65	11.3
60	8.1
84	21.7
67	13.4
125	10.6
132	5.7
9	15.6
18	11.3
55	23.7
75	8.7
150	16.1
104	20.7
135	11.6
137	20.8
164	11.9
76	6.9

Name: Sales, dtype: float64
Training set size: 160 rows
Testing set size: 40 rows

Feature Scaling -

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train_s = sc.fit_transform(x_train)
x_test_s = sc.transform(x_test)
print(x_train_s[:5])
```

```
[[-0.40424839 -1.02797963 -0.33767538]
 [ 0.32060772 -0.91956889 -1.16143931]
 [-1.27051084  0.25939783  0.25425079]
 [-1.04235941 -0.69597176 -0.57444585]
 [ 0.8791034  -1.38709018 -0.70762924]]
```

Query 1 -

What is the average amount spent on TV advertising in the dataset?

```
avg_tv = dataset['TV'].mean()
print("Average TV Spend:", avg_tv)
```

```
print("-----")
print("-----")
```

```
Average TV Spend: 147.0425
```

- Final Conclusion :

- Result: Average TV spend = 147.04 units

- Insights :

The company invests heavily in TV advertising compared to other channels. This reflects the importance of television as a mass-reach medium. The higher average spend suggests that management already views TV as the central focus of the advertising strategy.

- Recommendations :

Continue allocating a substantial portion of the budget to TV advertising, as it reaches broad audiences and is expected to strongly influence sales.

Query 2 -

What is the correlation between radio advertising expenditure and product sales?

```
import pandas as pd
```

```
Correlations = dataset[["Radio", "Sales"]].corr()
print(Correlations)
```

	Radio	Sales
Radio	1.000000	0.349651
Sales	0.349651	1.000000

- Final Conclusion :

- Result: Correlation = 0.35 which is a moderate positive correlation.

- Insights : Radio advertising shows a moderate positive relationship with sales. While not as strong as TV's impact, it still contributes meaningfully to sales performance.

- Recommendations : Maintain radio advertising as a supporting channel in the media. Since the correlation is moderate, it can be useful for targeting specific markets, but it should not replace TV as the primary focus.

Query 3 -

Which advertising medium has the highest impact on sales based on the dataset?

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# X = dataset[["TV", "Radio", "Newspaper"]]
# y = dataset["Sales"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

regressor = LinearRegression()
regressor.fit(X_scaled, y)
for feature, coef in zip(["TV", "Radio", "Newspaper"],
regressor.coef_):
    print(f"{feature}: {coef:.4f}")

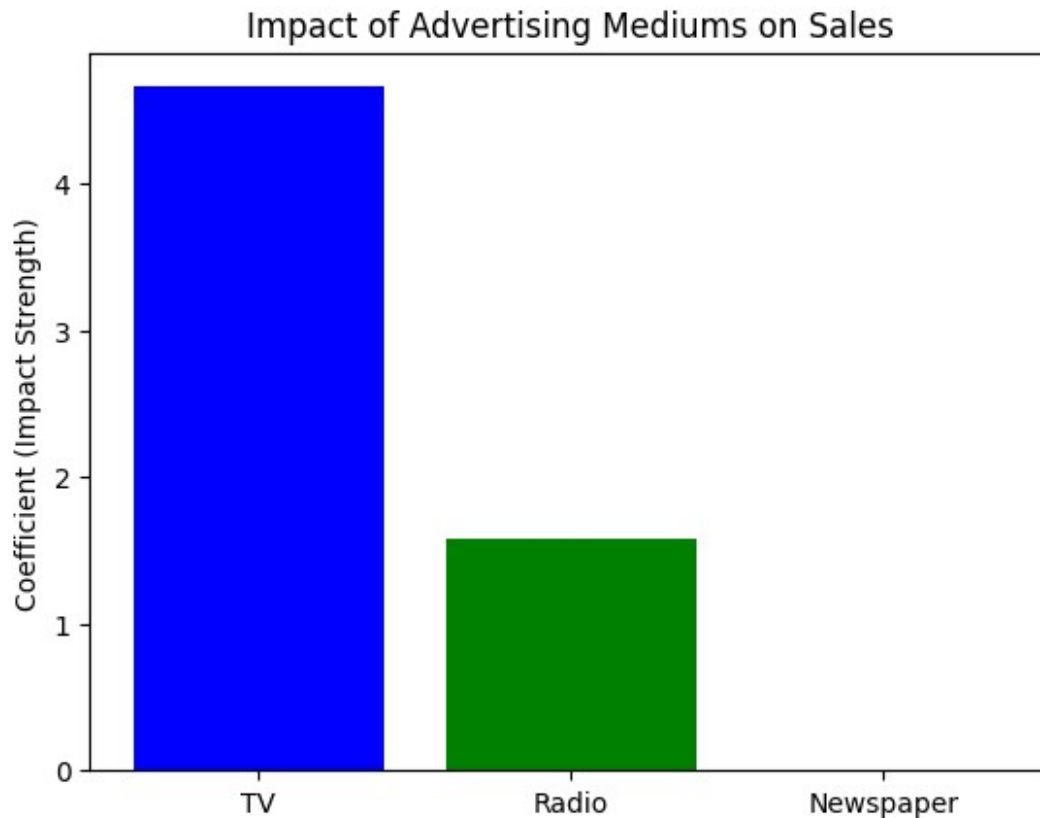
print("-----")

#for visualization
import matplotlib.pyplot as plt

features = ["TV", "Radio", "Newspaper"]
coeffs = [4.6627, 1.5849, 0.0070]

plt.bar(features, coeffs, color=["blue", "green", "red"])
plt.title("Impact of Advertising Mediums on Sales")
plt.ylabel("Coefficient (Impact Strength)")
plt.show()

TV: 4.6627
Radio: 1.5849
Newspaper: 0.0070
-----
--
```

- Final Verdict and Conclusion :

- Insights

TV advertising has the highest impact on Sales. Radio advertising has a moderate, positive effect. Newspaper advertising has negligible impact on Sales.

- Recommendations

Allocate more budget to TV campaigns to maximize sales. Maintain Radio as a supporting channel.

Reduce or eliminate Newspaper advertising spend and redirect resources to more effective channels.

Query 4 -

Plot a linear regression line that includes all variables (TV, Radio, Newspaper) to predict Sales, and visualize the model's predictions against the actual sales values.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

X = dataset[["TV", "Radio", "Newspaper"]]
```

```

y = dataset["Sales"]

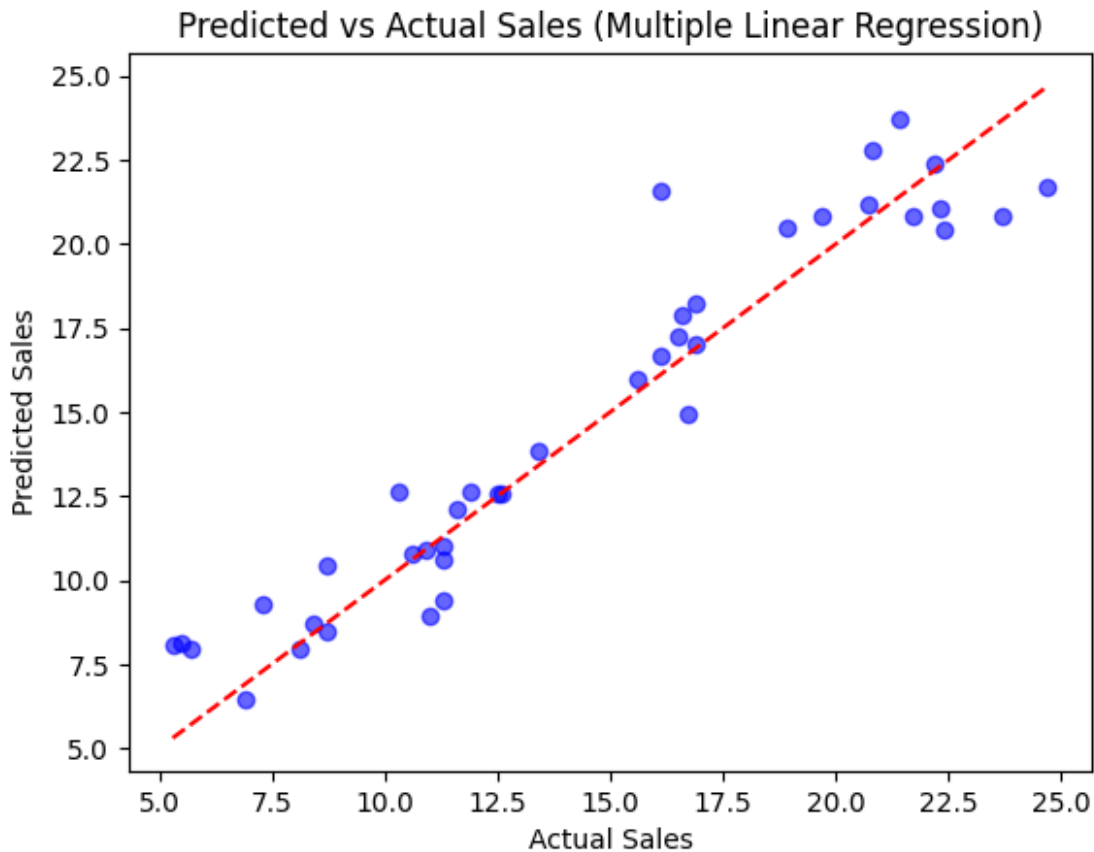
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

plt.scatter(y_test, y_pred, color="blue", alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
"r--") #actual vs predicted sales.
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Predicted vs Actual Sales (Multiple Linear Regression)")
plt.show()

```



- Final Verdict and Conclusions :

- Insights

The predicted values align closely with the actual sales values. This shows the regression model is a good fit for the data. Small deviations indicate there is still some unexplained variance, but overall the model is considered reliable.

- Recommendations

The model can be used to estimate sales based on advertising spend across TV, Radio, and Newspaper. Since the predictions are close to actuals, this regression approach can support budget allocation decisions.

Further improvements could be tested by including new advertising channels in future models.

Query 5 -

How would sales be predicted for a new set of advertising expenditures: 200 on TV, 40 on Radio, and 50 on Newspaper?

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X, y)

'''new_data = [[200, 40, 50]] throws a warning but still works '''
new_data = pd.DataFrame([[200, 40, 50]], columns=["TV", "Radio",
"Newspaper"])

predicted_sales = regressor.predict(new_data)
print("Predicted Sales:", predicted_sales[0])

Predicted Sales: 19.81151092944479
```

- Final Conclusion :

Based on the regression model, if the company spends 200 on TV, 40 on Radio, and 50 on Newspaper, the predicted Sales would be approximately 19.8 units.

Query 6 -

How does the performance of the linear regression model change when the dataset is normalized?

- Insights

Normalizing the dataset ensures that all advertising variables (TV, Radio, Newspaper) are measured on the same scale. While the overall predictive accuracy of the linear regression model remains almost identical with or without normalization, the interpretation of coefficients changes. After normalization, the coefficients represent the relative influence of each advertising medium in standardized units, making it clear that TV contributes the most, Radio has a

moderate impact, and Newspaper has almost negligible effect. This provides a clearer understanding of the comparative importance of each feature.

- Recommendations

For future analysis, normalization should be applied when the primary goal is interpretability of the coefficients and fair comparison across predictors. This will help decision-makers easily identify which advertising channel is most effective. However, if the focus is purely on prediction accuracy, normalization is optional since it does not significantly alter model performance. It is also advisable to normalize when adding new features with very different scales to maintain numerical stability and consistency in the model.

Query 7 -

What is the impact on the sales prediction when only radio and newspaper advertising expenditures are used as predictors?

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

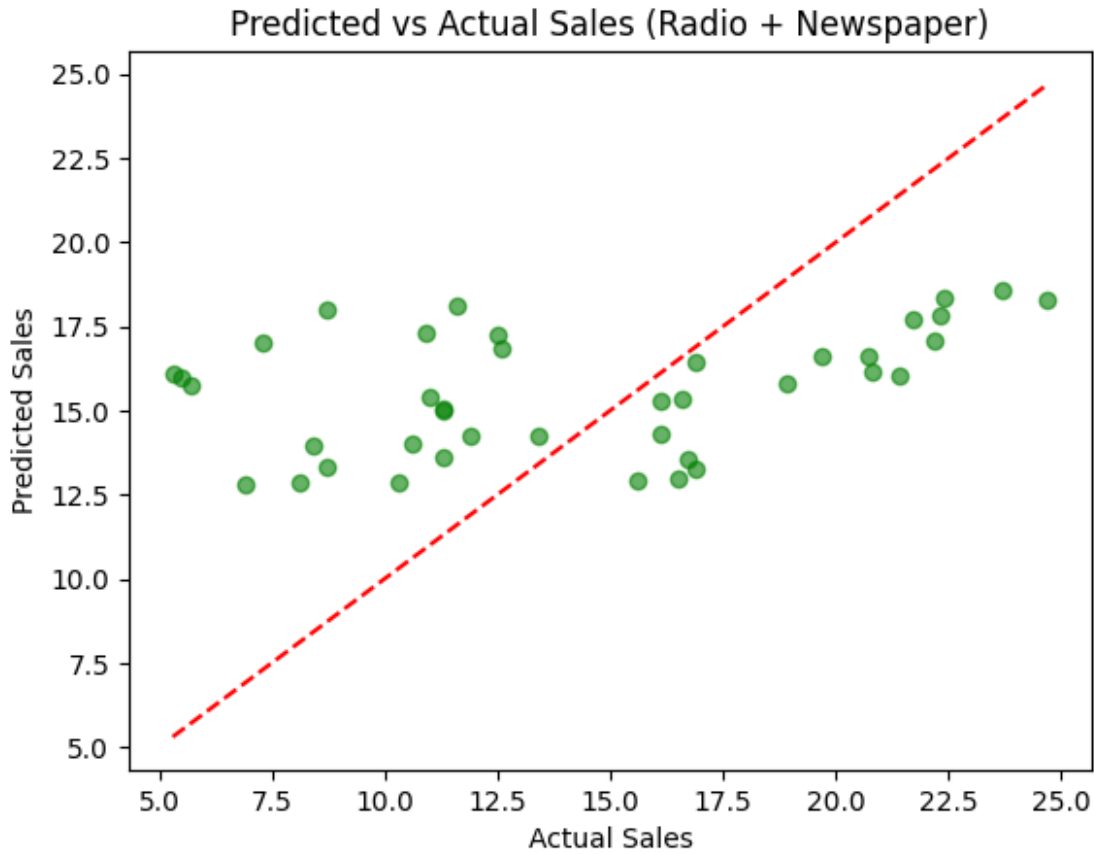
X = dataset[["Radio", "Newspaper"]]
y = dataset["Sales"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

plt.scatter(y_test, y_pred, color="green", alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
"r--")
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Predicted vs Actual Sales (Radio + Newspaper)")
plt.show()
```



- Final Conclusions :

- Insights

When only Radio and Newspaper are used as predictors, the regression model performs poorly compared to the full model. The predicted values are scattered and deviate significantly from the actual sales line, indicating high error and low explanatory power. This happens because TV advertising, the strongest predictor of sales, is excluded. Radio has a moderate impact, while Newspaper contributes very little, so the model cannot reliably capture sales variation.

- Recommendations

Do not rely solely on Radio and Newspaper for sales forecasting, as the model underperforms without TV data. Budget allocation decisions should prioritize TV advertising for accurate predictions and better return on investment, while Radio can play a supporting role. Newspaper spending should be minimized since it adds little predictive value.